Report On

SignSense: Deep CNN-Based Traffic Sign Recognition

Submitted in partial fulfillment of the requirements of the Course Project for Deep Learning in Semester VII of Final Year Artificial Intelligence & Data Science

By

Yash Rane (Roll No. 39)
Dishant Sasane (Roll No. 42)
Shayaan Shaikh (Roll No. 45)

Under the guidance

of

Mr. Manas Tambe



University of Mumbai

Vidyavardhini's College of Engineering & Technology

Department of Artificial Intelligence & Data Science



(A.Y. 2026-27)

Vidyavardhini's College of Engineering and Technology

Department of Artificial Intelligence & Data Science

CERTIFICATE

This is to certify that the project entitled "SignSense: Deep CNN-Based Traffic Sign Recognition" is a bonafide work of Shayaan Shaikh (Roll No. 45), Yash Rane (Roll No. 39), Dishant Sasane (Roll No. 42) submitted to the University of Mumbai in partial fulfillment of the requirement of the Course Project for Deep Learning in Semester VII of Final Year Artificial Intelligence & Data Science.

_____

Guide

Abstract

Traffic sign recognition is a key part of driver assistance systems and self-driving cars. This project, SignSense, classifies images of road traffic signs into 43 classes using deep learning on the German Traffic Sign Recognition Benchmark (GTSRB), which contains 39,209 training and 12,630 test images. We designed a compact convolutional neural network (CNN) trained from scratch on 32x32 images with data augmentation, dropout and batch normalization, and compared it with transfer learning using MobileNetV2 pre-trained on ImageNet. The custom CNN with batch normalization achieved 98.60% accuracy on the official test set using only 487,243 parameters. MobileNetV2 reached 88.30% after fine-tuning its top 60 layers, and only 66.75% with light fine-tuning. Error analysis shows that most mistakes involve triangular warning signs with small, similar symbols and rare classes. A Streamlit web application lets users upload a sign image and see the predicted sign, its confidence and the top-3 predictions.

Table of Contents

Chapter No.		Title	Page Number
1		**Introduction**	**1**
	1.1	Motivation	1
	1.2	Background	1
2		**Problem Statement**	**1**
	2.1	Objectives	1
	2.2	Dataset	2
3		**Proposed System**	**3**
	3.1	Block diagram, its description and working	3
	3.2	Module Description	4
4		**Implementation Result and Analysis**	**4**
	4.1	Implementation Screenshots	5
	4.2	Training and Test Results	5
	4.3	Comparison of Model A and Model B	6
	4.4	Error Analysis	7
5		**Code**	**8**
6		**Conclusion**	**10**
	6.1	Limitations and Future Scope	11
		References	**11**

1. Introduction

1.1 Motivation

Road traffic signs inform drivers about speed limits, dangers, priorities and mandatory directions. Missing or misreading a sign can cause accidents, especially at night, in bad weather or when the driver is tired. Advanced Driver Assistance Systems (ADAS) and self-driving cars therefore need to recognise traffic signs automatically, quickly and reliably.

The task is harder than it looks. Real photos vary in lighting, blur, viewing angle and distance, and many signs look alike: speed limits differ by a single digit, and many warning signs share the same red triangle and differ only in a small black symbol in the middle.

1.2 Background

Convolutional Neural Networks (CNNs) learn visual features such as edges, shapes and symbols directly from data, and have become the standard approach for image classification. The GTSRB benchmark [1] showed that machine learning methods can match human performance on traffic sign recognition. Transfer learning reuses a network pre-trained on a large dataset such as ImageNet; MobileNetV2 [2] is a lightweight network designed for mobile devices. Our models also use batch normalization [3], dropout [4] and the Adam optimizer [5].

This project builds a small CNN from scratch, compares it with MobileNetV2 transfer learning, analyses the errors, and provides a web demo that predicts the sign in an uploaded image.

2. Problem Statement

Build a deep learning system that takes an image of a road traffic sign and correctly classifies it into one of 43 classes (stop, yield, speed limits, turn signs, etc.), with high accuracy, proper evaluation and an easy-to-use demo.

2.1 Objectives

- Build and train a custom CNN from scratch that reaches at least 95% test accuracy.

- Build a second model using transfer learning (MobileNetV2) and compare it with the custom CNN.
- Evaluate both models with accuracy and loss curves, a confusion matrix, per-class precision, recall and F1-score, and analysis of wrong predictions.
- Build a web demo where a user uploads a sign image and gets the predicted sign and confidence.
- Produce clean, documented and reproducible code.

2.2 Dataset

We use the German Traffic Sign Recognition Benchmark (GTSRB) [1] from Kaggle [6]. It contains real photos taken from a moving car, cropped around the sign.

- **Classes:** 43 sign types (speed limits, prohibitions, warnings, mandatory directions).
- **Size:** 39,209 training images and 12,630 test images; image sides range from 25 to 266 pixels, and many are dark or blurry.
- **Class imbalance:** from 210 images (Speed limit 20 km/h) to 2,250 images (Speed limit 50 km/h) per class, with a mean of 912 (Fig. 2.1).
- **Split:** 80% training (31,367 images) and 20% validation (7,842 images), stratified by class. The official test set is used only for the final evaluation.

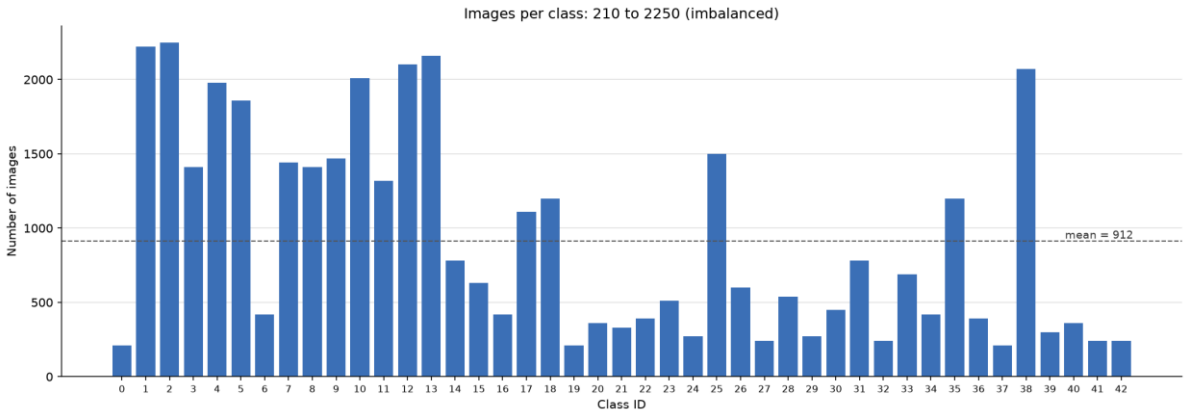


Fig. 2.1: Number of training images per class

3. Proposed System

3.1 Block diagram, its description and working

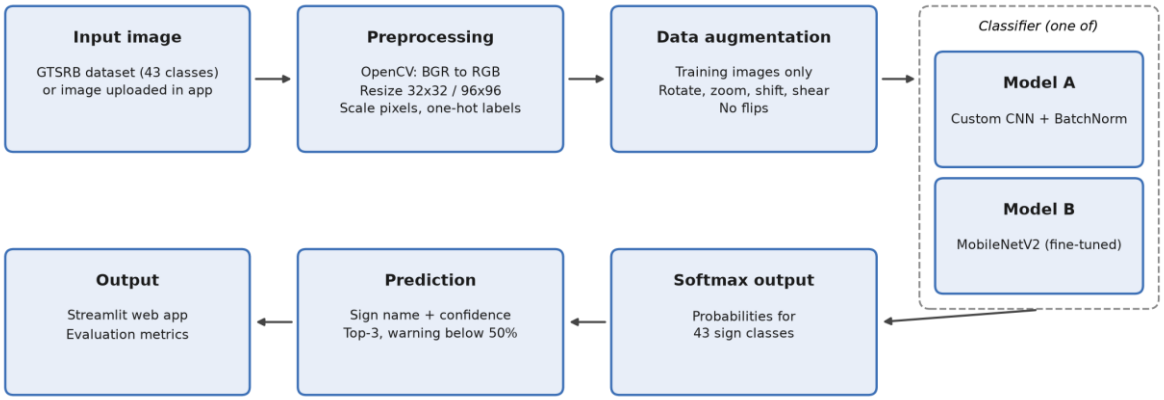


Fig. 3.1: Block diagram of SignSense

Working: An input image (from the dataset or uploaded in the app) is read with OpenCV, converted from BGR to RGB and resized to 32x32 pixels for the custom CNN or 96x96 pixels for MobileNetV2. Pixel values are scaled (divided by 255 for the CNN; scaled to -1..1 for MobileNetV2) and labels are one-hot encoded.

During training only, images are augmented with random rotation (10 degrees), zoom (15%), shift (10%) and shear. Images are never flipped, because flipping changes the meaning of a sign: "Turn left ahead" would become "Turn right ahead" (Fig. 3.2).

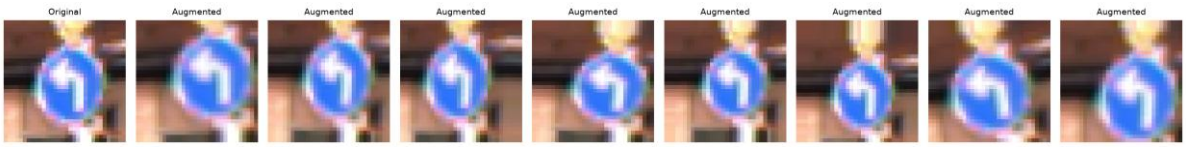


Fig. 3.2: A "Turn left ahead" sign and eight augmented versions (no flips)

The image is then passed to the classifier. **Model A** is our custom CNN: two blocks of two 3x3 convolutions (32 and 64 filters) with batch normalization, max pooling and dropout (0.25), followed by Flatten, Dense(256), Dropout(0.5) and a 43-way softmax layer (487,243 parameters). **Model B** is MobileNetV2 pre-trained on ImageNet with a new head (GlobalAveragePooling, Dropout(0.3), Dense(43)); it is first trained with the base frozen and then fine-tuned by unfreezing the top 60 layers (2,313,067 parameters).

The softmax layer outputs a probability for each of the 43 classes. The class with the highest probability is the prediction and its probability is the confidence. The web app shows the sign name, the confidence, the top-3 predictions and a warning when the confidence is below 50%.

3.2 Module Description

The code is organised as a small Python package. Each module has one clear job:

Table 3.1: Project modules

Module	Description
config.py	All paths and hyperparameters (image size, batch size, epochs, learning rates).
data_loader.py	Loads images with OpenCV (BGR to RGB, resize) and labels; caches them as .npz.
preprocess.py	Normalisation, one-hot labels, stratified 80/20 split, augmentation, tf.data pipeline.
models.py	build_custom_cnn() for Model A and build_mobilenet() for Model B.
train.py	Trains a model with EarlyStopping, ModelCheckpoint and ReduceLROnPlateau; saves history.
evaluate.py	Test accuracy, training curves, confusion matrix, per-class report, wrong predictions.
compare.py	Comparison table and chart of all trained models.
predict.py	predict_image(): predicts the sign in one image with the training preprocessing.
app/app.py	Streamlit web app: upload, model choice, prediction, confidence, top-3 chart.

4. Implementation Result and Analysis

The project was implemented in Python with TensorFlow / Keras, OpenCV, NumPy, pandas, scikit-learn, Matplotlib, Seaborn and Streamlit. The custom CNN was trained on a laptop CPU (Python 3.11, TensorFlow 2.16) and MobileNetV2 on a Google Colab T4 GPU.

4.1 Implementation Screenshots

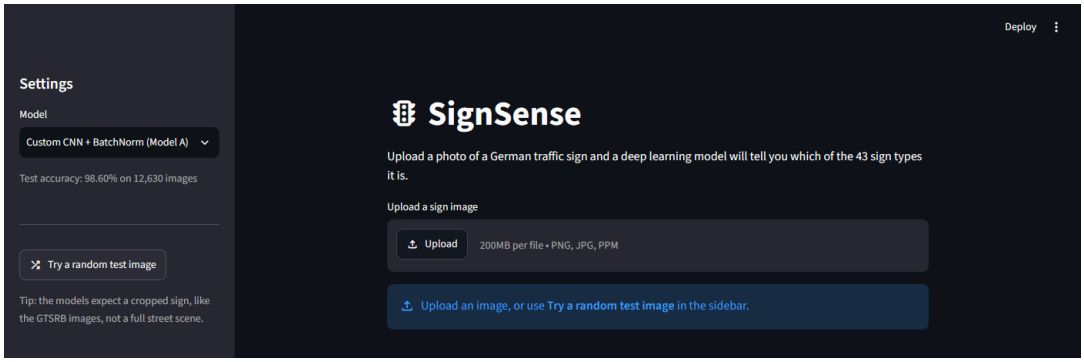


Fig. 4.1: SignSense web app: home page

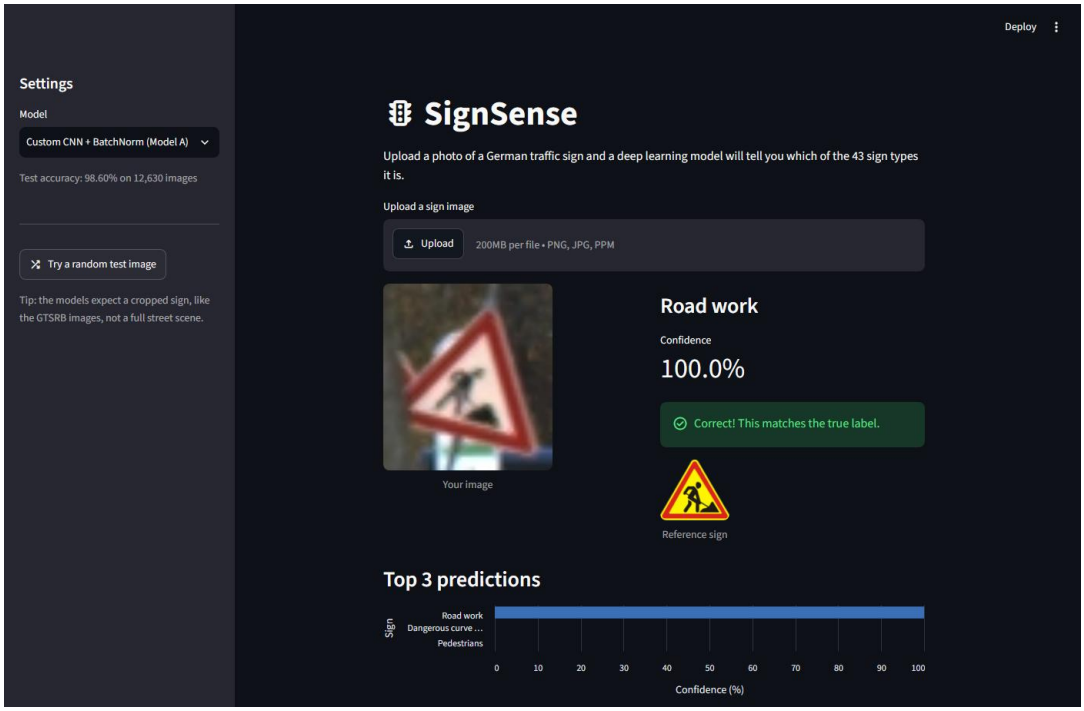


Fig. 4.2: SignSense web app: prediction with confidence and top-3 chart

4.2 Training and Test Results

Table 4.1: Training settings

Setting	Model A: Custom CNN	Model B: MobileNetV2
Input size	32 x 32 x 3	96 x 96 x 3
Optimizer, learning rate	Adam, 0.001	Adam, 0.001 then 0.0001
Epochs, batch size	up to 20, 64	10 + 15 (fine-tuning), 64
Training time	12.5 min (CPU)	8.9 min (T4 GPU)

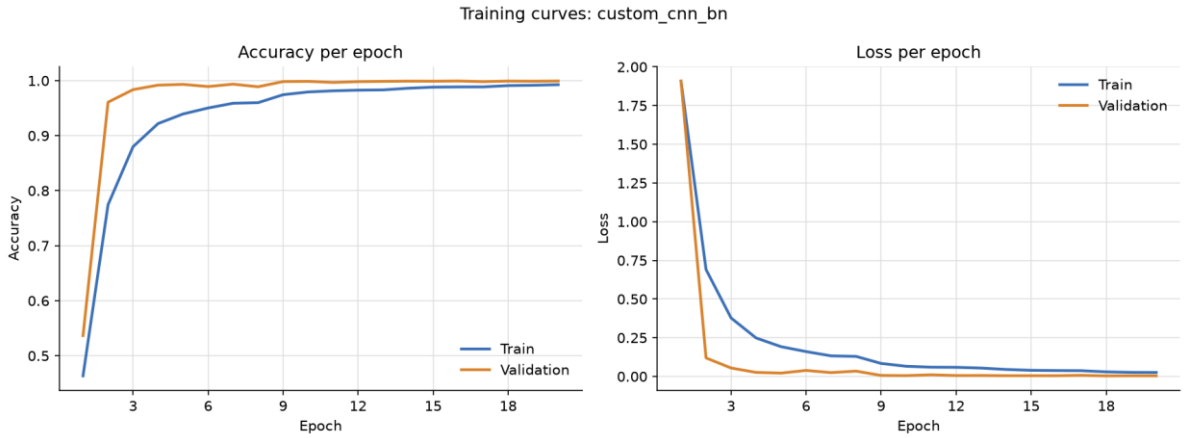


Fig. 4.3: Training and validation accuracy and loss of Model A

Model A reached **98.60% accuracy on the official test set** (177 wrong out of 12,630), above the 95% target. Validation accuracy (99.92%) is higher than test accuracy because the training data contains "tracks" of about 30 near-identical photos of each physical sign, so a random split puts similar images in both training and validation. Training accuracy stays slightly below validation accuracy because training images are augmented and dropout is active only during training. Adding batch normalization to the baseline CNN raised test accuracy from 98.05% to 98.60% for only 768 extra parameters.

4.3 Comparison of Model A and Model B

Table 4.2: Comparison on the official test set (12,630 images)

Model	Parameters	Test accuracy	Macro-F1	File size
Custom CNN (baseline)	486,475	98.05%	0.963	5.9 MB
Custom CNN + BatchNorm (Model A)	487,243	98.60%	0.973	5.9 MB
MobileNetV2, light fine-tuning	2,313,067	66.75%	-	-
MobileNetV2, fine-tuned (Model B)	2,313,067	88.30%	0.842	26.0 MB

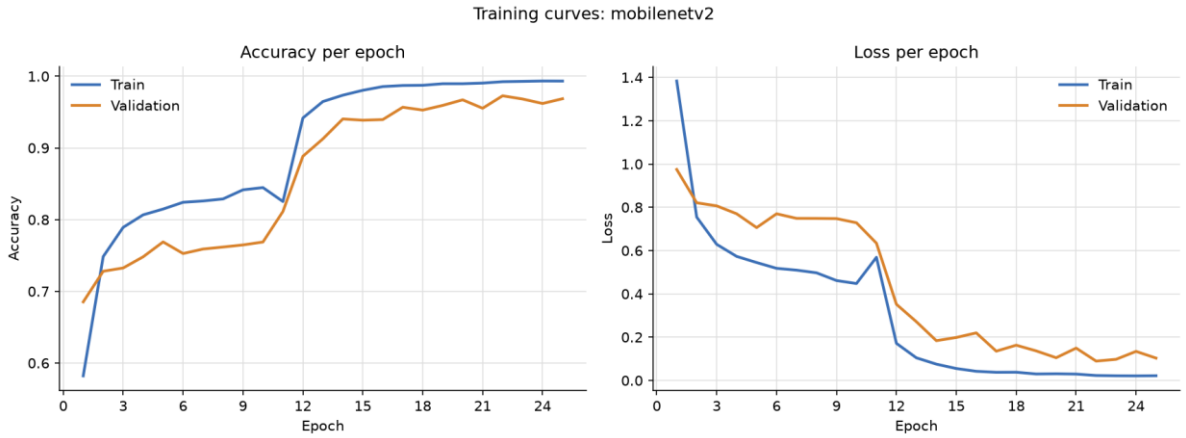


Fig. 4.4: MobileNetV2 training: fine-tuning starts at epoch 11

The small custom CNN is more accurate than MobileNetV2 while being about 5 times smaller. MobileNetV2 was pre-trained on large, detailed ImageNet photos, while GTSRB images are tiny and blurry. Its GlobalAveragePooling layer also discards the position of features, so left and right signs are confused ("Go straight or right" was predicted as "Go straight or left" 33 times). Fine-tuning was essential: with the base frozen, validation accuracy stalled at about 77%, and it jumped to 97% once the top 60 layers were unfrozen (Fig. 4.4).

4.4 Error Analysis

Table 4.3: Five classes with the lowest F1-score (Model A)

Class	F1	Recall	Main confusion
27 Pedestrians	0.66	50%	predicted as Children crossing (25 of 60)
22 Bumpy road	0.86	75%	predicted as Road work (28 of 120)
40 Roundabout mandatory	0.89	99%	Priority road predicted as Roundabout (19)
28 Children crossing	0.92	100%	receives the Pedestrians errors (25)
21 Double curve	0.93	100%	General caution predicted as Double curve (13)

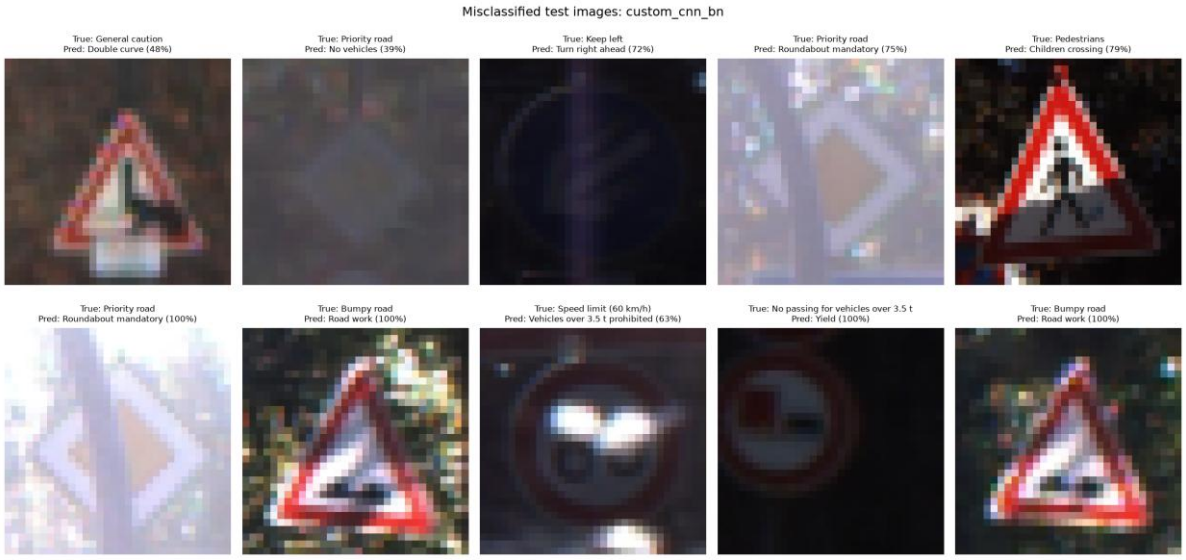


Fig. 4.5: Examples of misclassified test images (Model A)

Most errors are between triangular warning signs with the same red border, whose only difference is a small black symbol that is just a few pixels wide at 32x32. Several of these classes are rare (Pedestrians has only 240 training images), and many wrong images are very dark, blurred or overexposed.

5. Code

The most important parts of the code are shown below. The full code is in the project repository and in the Colab notebook.

```python
# src/models.py

def _conv_block(model, filters, use_batchnorm):
    for _ in range(2):
        model.add(layers.Conv2D(filters, (3, 3), activation="relu"))
        if use_batchnorm:
            # BatchNorm keeps activations at a stable scale, which often speeds up training
            model.add(layers.BatchNormalization())
    model.add(layers.MaxPooling2D(pool_size=(2, 2)))   # halve width and height
    model.add(layers.Dropout(0.25))                     # randomly drop 25% to reduce overfitting


def build_custom_cnn(use_batchnorm=False):
    name = "custom_cnn_bn" if use_batchnorm else "custom_cnn"
    model = models.Sequential(name=name)
    model.add(layers.Input(shape=(config.IMG_SIZE, config.IMG_SIZE, config.NUM_CHANNELS)))

    # Feature extractor: learns edges, shapes, then sign parts
    _conv_block(model, 32, use_batchnorm)
    _conv_block(model, 64, use_batchnorm)

    # Classifier: turns the learned features into 43 class probabilities
    model.add(layers.Flatten())
    model.add(layers.Dense(256, activation="relu"))
    model.add(layers.Dropout(0.5))
    model.add(layers.Dense(config.NUM_CLASSES, activation="softmax"))

    _compile(model, config.LEARNING_RATE)
    return model
```

Fig. 5.1: Model A, custom CNN (models.py)

```python
# src/models.py

def build_mobilenet():
    size = config.MOBILENET_IMG_SIZE
    base_model = MobileNetV2(weights="imagenet", include_top=False,
                             input_shape=(size, size, config.NUM_CHANNELS))
    base_model.trainable = False   # freeze: keep the ImageNet features unchanged at first

    inputs = layers.Input(shape=(size, size, config.NUM_CHANNELS))
    # training=False keeps the base's BatchNorm layers in inference mode,
    # which matters when we unfreeze some layers for fine-tuning later
    features = base_model(inputs, training=False)
    x = layers.GlobalAveragePooling2D()(features)   # average each feature map to one number
    x = layers.Dropout(0.3)(x)
    outputs = layers.Dense(config.NUM_CLASSES, activation="softmax")(x)

    model = models.Model(inputs, outputs, name="mobilenetv2")
    _compile(model, config.LEARNING_RATE)
    return model, base_model


def unfreeze_top_layers(model, base_model, num_layers=config.FINE_TUNE_LAYERS):
    base_model.trainable = True
    for layer in base_model.layers[:-num_layers]:
        layer.trainable = False
    # A tiny learning rate nudges the pre-trained weights instead of destroying them
    _compile(model, config.FINE_TUNE_LR)
```

Fig. 5.2: Model B, MobileNetV2 transfer learning (models.py)

```python
# src/train.py

def get_callbacks(model_path, best_val_accuracy=None):
    return [
        # Stop when validation loss stops improving, and go back to the best epoch
        EarlyStopping(monitor="val_loss", patience=config.EARLY_STOP_PATIENCE,
                      restore_best_weights=True, verbose=1),
        # Save the model every time validation accuracy reaches a new best
        ModelCheckpoint(str(model_path), monitor="val_accuracy", save_best_only=True,
                        initial_value_threshold=best_val_accuracy, verbose=1),
        # Halve the learning rate when validation loss gets stuck
        ReduceLROnPlateau(monitor="val_loss", factor=config.REDUCE_LR_FACTOR,
                          patience=config.REDUCE_LR_PATIENCE, verbose=1),
    ]


def train_custom_cnn(use_batchnorm=False):
    set_seeds()
    _make_output_folders()

    X_train, X_val, _, y_train, y_val, _ = prepare_data()   # test set is NOT used here
    model = build_custom_cnn(use_batchnorm)
    model.summary()

    model_path = config.MODELS_DIR / f"{model.name}.keras"
    # The augmenter creates new randomly rotated/zoomed/shifted batches every epoch
    train_batches = create_augmenter().flow(X_train, y_train,
                                            batch_size=config.BATCH_SIZE, seed=config.RANDOM_SEED)

    start_time = time.time()
    history = model.fit(
        train_batches,
        validation_data=(X_val, y_val),   # validation images are NOT augmented
        epochs=config.EPOCHS,
        callbacks=get_callbacks(model_path),
    )
    train_time = time.time() - start_time
    print(f"Training took {train_time / 60:.1f} minutes. Best model saved to {model_path}")

    save_history(history.history, train_time, config.RESULTS_DIR / f"history_{model.name}.json")
    return model, history
```

Fig. 5.3: Training with callbacks (train.py)

```python
# src/predict.py

def preprocess_for_model(rgb_image, model_name):
    is_mobilenet = model_name.startswith("mobilenet")
    size = config.MOBILENET_IMG_SIZE if is_mobilenet else config.IMG_SIZE
    resized = cv2.resize(rgb_image, (size, size), interpolation=cv2.INTER_AREA)
    batch = resized[np.newaxis].astype("float32")   # shape (1, size, size, 3)

    if is_mobilenet:
        return preprocess_input(batch)              # scale to [-1, 1]
    return batch / 255.0                            # scale to [0, 1]


def predict_image(image, model_name=config.MODEL_A_NAME):
    sign_names = load_sign_names()
    batch = preprocess_for_model(to_rgb_array(image), model_name)
    probabilities = get_model(model_name).predict(batch, verbose=0)[0]

    # Indices of the 3 highest probabilities, largest first
    top_ids = np.argsort(probabilities)[::-1][:config.TOP_K]
    top3 = [(int(i), sign_names[int(i)], float(probabilities[i])) for i in top_ids]

    class_id, class_name, confidence = top3[0]
    return class_id, class_name, confidence, top3
```

Fig. 5.4: Predicting a single image (predict.py)

6. Conclusion

We built SignSense, a traffic sign recognition system for the 43 GTSRB classes. A compact custom CNN with data augmentation (without flips), dropout and batch normalization reached

98.60% test accuracy with fewer than 0.5 million parameters, above the 95% target. MobileNetV2 transfer learning reached 88.30% after fine-tuning, showing that pre-trained ImageNet features do not transfer well to tiny, low-resolution sign images and that fine-tuning is essential. The remaining errors come mainly from similar triangular warning signs, rare classes and poor image quality. A Streamlit web app demonstrates the model on uploaded images.

6.1 Limitations and Future Scope

- The models classify already cropped signs; real-time detection in full camera frames could be added with an object detector such as YOLO.
- Video input from a dashcam, combining predictions over several frames.
- Training on more datasets, such as Indian traffic signs, to handle other countries.
- Deployment on mobile and edge devices with TensorFlow Lite; our model is only about 6 MB.
- Higher accuracy through cropping to the sign's bounding box, class weights for rare classes and higher input resolution.

References

[1] J. Stallkamp, M. Schlipsing, J. Salmen and C. Igel, "Man vs. computer: Benchmarking machine learning algorithms for traffic sign recognition," Neural Networks, vol. 32, pp. 323-332, 2012.

[2] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov and L.-C. Chen, "MobileNetV2: Inverted residuals and linear bottlenecks," in Proc. IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), 2018, pp. 4510-4520.

[3] S. Ioffe and C. Szegedy, "Batch normalization: Accelerating deep network training by reducing internal covariate shift," in Proc. International Conference on Machine Learning (ICML), 2015, pp. 448-456.

[4] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever and R. Salakhutdinov, "Dropout: A simple way to prevent neural networks from overfitting," Journal of Machine Learning Research, vol. 15, pp. 1929-1958, 2014.

[5] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in Proc. International Conference on Learning Representations (ICLR), 2015.

[6] Kaggle, 'GTSRB - German Traffic Sign Recognition Benchmark'. [Online]. Available: https://www.kaggle.com/datasets/meowmeowmeowmeowmeow/gtsrb-german-traffic-sign. [Accessed: 19- Oct- 2026].